Evaluating A Large Language Model Approach in Unadmitted Technical Debt Detection

Miguel de Oliveira Rodrigues
Unidade Acadêmica de Sistemas e Computação (UASC)
Campina Grande, Brasil
miguel.rodrigues@ccc.ufcg.edu.br

João Arthur Brunet
Curso de Ciência da Computação (CCC)
Campina Grande, Brasil
joao.arthur@computacao.ufcg.edu.br

Abstract

"In recent years, research has emerged on identifying Technical Debt (TD) that remains unacknowledged by developers. Specifically, Yu et al. [10] developed a machine learning approach to detect code snippets with "Unadmitted Technical Debt", that is, code snippets that have similar patterns identified in Self-Admitted Technical Debt cases. However, their approach relies solely on code metrics, thereby omitting the functional context of the code for their model. This work evaluates the efficacy of providing the entire code snippet (excluding comments) to a Large Language Model (LLM) and compares its performance against their metric-based approach.

To achieve this, we conducted an experiment using Yu et al.'s dataset[1], utilizing Qwen2.5-7B-Instruct to identify the presence of TD in each code snippet, and compared the outcomes to the LiteM results reported in their paper.

Through this experiment, we found that, despite receiving richer context from the raw code, the chosen LLM underperformed in TD identification compared to the baseline model.

ACM Reference Format:
Miguel de Oliveira Rodrigues and João Arthur Brunet. 2026. Evaluating A Large Language Model Approach in Unadmitted Technical Debt Detection. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Technical Debt (TD), in software development context, is a metaphor that describes suboptimal or poorly designed code that may negatively impact software maintainability and increase development costs in the long term [2–4]. A specific subset, called "Self-Admitted Technical Debt" (SATD), refers to instances where developers explicitly acknowledge the presence of TD via text artifacts, such as source code comments or GitHub issues.

Extensive research has demonstrated how these suboptimal solutions compromise software quality and maintainability [**?**]. Consequently, numerous studies have explored methodologies to identify [5, 9–11] and manage [7, 8] TD, aiming to mitigate future development costs.

Recently, in 2025, Yu et al. [10] introduced a new concept related to TD: "Unadmitted Technical Debt" (UTD). This concept refers to code snippets that developers have not indicated as containing TD, but that exhibit patterns similar to code present in SATD cases. Thus, they made four contributions to this newly introduced topic:

(1) Proposed an automated framework for UTD annotation.
(2) Constructed a large TD dataset of multi-granularity code snippets.
(3) Proposed an approach named LiteM for detecting UTD solely based on code metrics.
(4) Proposed an approach named LiteMC for detecting UTD in real-world scenarios.

Specifically, Yu et al. [10] evaluated their proposed TD detection model, LiteM, against two baseline models: TEDIOUS by Maldonado et al. [6] and SATDID by Alhefdhi et al. [1], using their multi-granularity code snippet dataset.

LiteM relies solely on source code metrics, such as Lines-of-Code (LoC) and Lack of Cohesion of Methods (LCoM), to identify the presence of TD, while TEDIOUS incorporates static analysis warnings like "AvoidReassigningParameters". Crucially, neither of these models receives any semantic context regarding the code's actual functionality.

SATDID, on the other hand, processes the source code parsed as Abstract Syntax Trees (ASTs), thereby preserving some structural information of the code's semantics. However, because it was trained exclusively using conditional-level analysis, its evaluation was limited to the block granularity level within Yu et al.'s dataset. Consequently, the performance of a model capable of leveraging the full semantics of the source code to detect TD across all granularity levels has yet to be evaluated.

This work expands Yu et al.'s experiment by evaluating the performance of a Large Language Model (Qwen2.5-7B-Instruct) in TD detection compared to the metric-based model, LiteM. This analysis is crucial because it contrasts two fundamentally different paradigms for the same task: while LiteM relies exclusively on engineered code metrics, LLMs harness the raw source code as context to classify the code.

To achieve this, our work adopts an experimental setup aligned with Yu et al. [10]: we utilize the identical dataset and evaluation metrics to assess our model, shifting the input from code metrics to raw source code. Besides, to ensure a fair comparison, all source code comments are removed from the code snippets, as the dataset contains comments that explicitly indicate SATD.

[1] https://github.com/HduDBSI/Dataset4TD

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Methodology

This section presents the methodology applied to evaluate the performance of the chosen Large Language Model, Qwen2.5-7B-Instruct[2], in Technical Debt (TD) detection.

2.1 Research Question

This experiment intends to answer the following research question:

> **RQ:** How does a Large Language Model (Qwen2.5-7B-Instruct) compare in performance to the model proposed by Yu et al. [10] (LiteM) in detecting Technical Debt in code snippets without the aid of comments?

To answer this question, we conduct an experiment using Yu et al.'s dataset and evaluation metrics and compare the results to the ones reported in their paper.

2.2 Dataset Description

Yu et al. [10] built a large dataset to evaluate models in the task of detecting code snippets with technical debt. The main characteristics of this dataset are:

a. It contains 364,932 code snippets from 18 open-source Java GitHub projects.
b. Each code snippet has a label classifying it as containing UTD or not.
c. It is divided into four granularity levels: file, class, method, and block.
d. It contains one column for each one of the extracted code metrics per code snippet. Each granularity level has a different set of code metrics that the authors were able to extract.

For our experiment, only the source code and its label (whether it contains TD or not) were used. To ensure a fair comparison to the LiteM model, the comments from the source code were removed to prevent passing any SATD comments to the LLM.

2.3 Resource Limitations

Evaluating a dataset of this magnitude (364,932 code snippets) presents a significant computational challenge when using a Large Language Model. Specifically, a single experimental run requires approximately 45 hours of continuous local inference on our available machine (equipped with 4x NVIDIA Tesla V100 SXM2 16GB (NVLink2)), making the inclusion of multiple models computationally expensive. The same constraint applies to running multiple inferences with the same model or testing different prompting strategies, particularly those that increase the number of input and/or output tokens, such as the Chain-of-Thought and few-shot strategies. Thus, we leave testing other models and alternative prompting strategies for future work.

2.4 Model Selection

While testing multiple LLMs would benefit this analysis, due to the resource limitations discussed in Section 2.3, this experiment relies on a single model. Thus, we selected Qwen2.5-7B-Instruct as a representative open-source model for this evaluation.

[2]https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

2.5 Classification with the LLM

Each code snippet from the dataset by Yu et al. [10] was provided to Qwen2.5-7B-Instruct for classification. The model was instructed to output "yes" if TD was present, and "no" otherwise. Since LLMs pose the risk of hallucination, the following measures were taken to prevent the model from answering with invalid sentences:

1. The limitation on the answer being only "yes" or "no" was specified in the system prompt to the model.
2. The model temperature parameter was set to 0 to ensure maximum determinism in the model's classifications.
3. The number of maximum new tokens was set to 2. This way, the Qwen2.5 model would be able to answer "yes" or "no" and end the completion with the "`<|end_of_text|>`" special token while preventing it from generating any extra explanations.

Additionally, all the outputs were verified using a Python script to guarantee that all generated answers fell into the yes-no constraint. Each answer with an invalid token was considered wrong, and the number of cases in which this happened is reported.

2.6 Prompting

The prompt used to instruct the Qwen2.5 model was structured following the official formatting guidelines provided in the Meta developer documentation[3].

The model was evaluated by processing one code snippet at a time. Furthermore, due to the limitations discussed in Section 2.3, a zero-shot approach was employed for prompting.

2.7 Evaluation Metrics

To evaluate the model, the same evaluation metrics applied by Yu et al. [10] were used in this experiment: Precision (P), Recall (R), F1-Score (F1), and Matthews Correlation Coefficient (MCC). Furthermore, the results were reported separately by granularity level and project, just as in Yu et al.'s paper [10]. This way, we were able to make a direct comparison with their original findings.

For these metrics, we classified the code snippets with TD as the "positive" class, while the ones without TD as the "negative" class.

Note that while Yu et al. also reported the Area Under the ROC Curve (AUC), this metric is omitted from our evaluation. Because our zero-shot prompting strategy configures the LLM to generate discrete text tokens ("yes" or "no") with a temperature of 0 for maximum determinism, the model does not output the continuous prediction probabilities or log-probabilities required to compute a standard AUC curve.

2.8 Statistical Hypothesis Testing and Confidence Intervals

To further evaluate the performance gap between the two approaches, we construct 95% confidence intervals (CIs) for the mean F1-Scores across all 18 subject projects using Student's t-distribution. We focus this analysis on the F1-Score as it was the primary metric used in the baseline evaluation by Yu et al. [10].

[3]https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_1/

To complement the interval estimation, we define the null hypothesis (H_0) as the assumption that there is no statistically significant difference between the F1-Scores of Qwen2.5-7B-Instruct and LiteM across the 18 projects. The alternative hypothesis (H_1) posits a significant difference exists. Before testing, a Shapiro-Wilk test is applied to verify the normality of the F1-Score distributions. If normality is confirmed ($p \geq 0.05$), a paired Student's t-test is used to compute the final significance; otherwise, the non-parametric Wilcoxon signed-rank test is applied.

2.9 Execution Environment

The evaluation experiments were executed on an server powered by an IBM Power9 processor architecture with 512 GB of system RAM. Local model inference was accelerated using 4× NVIDIA Tesla V100 SXM2 16GB GPUs connected via NVLink2. To maximize throughput and manage memory efficiently across the 364,932 code snippets, we deployed the model using the vLLM (v0.23.0) execution engine. All pipeline scripts for data preprocessing, comment stripping, and output parsing were implemented in Python.

3 Results and Discussion

This section presents the analysis derived from the results obtained in the experiment. Table 1 presents a comparison between the evaluation results for both the LLM (Qwen2.5-7B-Instruct) and the baseline model, LiteM. This section is divided into two parts, focusing first on the overall performance metrics and subsequently on the analysis of model hallucinations.

3.1 Overall Performance

Looking at Table 1, it is evident that LiteM outperforms the LLM in most metrics and granularity levels across nearly all scenarios. On average, the most significant performance gap is observed in the Matthews Correlation Coefficient (MCC). Because the dataset is heavily imbalanced, MCC serves as the most robust metric to compare these classifiers, since it accounts for both classes equally. Therefore, these results suggest that LiteM—despite receiving no information regarding code semantics—is a superior classifier for Technical Debt compared to the selected LLM. The performance gap is widest at the method-level granularity: while LiteM achieves its highest MCC at this granularity level (0.38), Qwen2.5-7B-Instruct performs at its lowest (0.07). The LLM also receives this low score at the block-level granularity, suggesting increased difficulty for the model when detecting TD in finer-grained code snippets.

This low performance from the LLM can be explained by the following factors:

- **Identification of Irrelevant Technical Debt:** The LLM generates an excessive number of false positives (as indicated by its low precision scores) because it frequently flags general code quality issues or minor stylistic choices that do not constitute actual Technical Debt within this scope. It struggles to distinguish meaningful debt from minor code anomalies. This behavioral pattern is explicitly illustrated in Listing 1 and Listing 2, which show an example where the model incorrectly labels a simple return and a common string manipulation as technical debt.
- **Context Limitations:** The model struggles significantly with finer-grained code snippets, such as methods and blocks, due to a severe lack of surrounding architectural context.
- **Misalignment with Dataset Ground Truth:** The dataset's ground truth relies strictly on developer comments to confirm the presence of TD. Because the LLM evaluates uncommented code based on its own training data, it often identifies code issues that do not align with the developer's admissions, further degrading its classification accuracy.

```
return true;
```

Listing 1: Simple return statement incorrectly labeled as Technical Debt (False Positive).

```
String description = "transfer of " + artifacts.size() +
    " files";
```

Listing 2: Common dynamic string construction incorrectly labeled as Technical Debt (False Positive).

3.2 Statistical Significance and Confidence Intervals

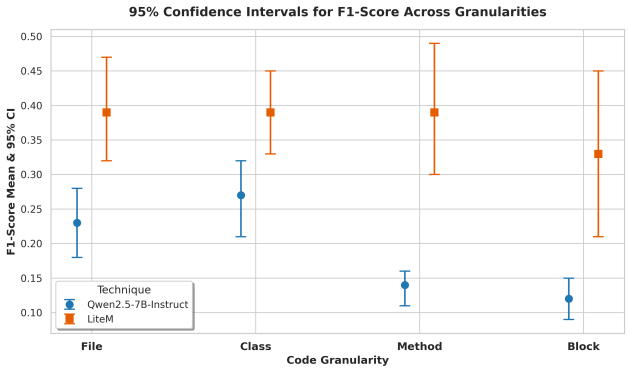

Figure 1: 95% Confidence Interval (CI) comparison of F1-Scores between Qwen2.5-7B-Instruct and LiteM across the 18 evaluated projects.

The statistical analysis reveals that LiteM's performance advantage over the LLM is statistically significant across all evaluated granularities. As illustrated in Figure 1, there is no overlap between the 95% confidence intervals of Qwen2.5-7B-Instruct and LiteM at any level (File, Class, Method, or Block). Even at the Class level, where the performance gap is narrowest, the lower bound of LiteM (≈ 0.33) remains strictly above the upper bound of the LLM (≈ 0.32).

To formally validate this divergence according to our methodology, we performed the Shapiro-Wilk normality test on the F1-scores. For the File granularity, both models satisfied the normality assumption ($p \approx 0.34$ for Qwen2.5 and $p \approx 0.69$ for LiteM), justifying a parametric approach. Consequently, a paired Student's t-test confirmed that LiteM outperforms at the File level and the difference is highly significant ($p < 0.001$, raw $p \approx 7.7 \times 10^{-5}$).

Table 1: Comprehensive evaluation results comparing Qwen2.5-7B-Instruct and LiteM across four distinct code granularities (File, Class, Method, and Block) on four subject projects. Performance is measured using Precision (P), Recall (R) and Matthews Correlation Coefficient (MCC). The highest values for each metric within a project and granularity are highlighted in bold.

Project	Technique	File				Class				Method				Block			
		P	R	F1	MCC	P	R	F1	MCC	P	R	F1	MCC	P	R	F1	MCC
antlr4	Qwen2.5-7B-Instruct	0.23	**0.51**	0.32	0.21	0.24	**0.81**	0.37	0.26	0.06	**0.38**	0.11	0.06	**0.04**	**0.33**	**0.08**	**0.06**
	LiteM	**0.39**	0.31	**0.34**	**0.27**	**0.43**	0.35	**0.38**	**0.29**	**0.28**	0.21	**0.23**	**0.21**	0.04	0.12	0.06	0.03
dbeaver	Qwen2.5-7B-Instruct	0.10	**0.58**	0.17	0.15	0.10	**0.82**	0.18	0.17	0.06	**0.39**	0.11	0.04	0.07	**0.40**	0.11	0.07
	LiteM	**0.28**	0.24	**0.25**	**0.22**	**0.29**	0.32	**0.30**	**0.26**	**0.16**	0.26	**0.20**	**0.16**	**0.44**	0.12	**0.14**	**0.18**
elasticsearch	Qwen2.5-7B-Instruct	0.13	**0.56**	0.21	0.15	0.16	**0.73**	0.27	0.16	0.08	**0.44**	0.14	0.11	0.07	**0.40**	0.11	0.08
	LiteM	**0.38**	0.36	**0.36**	**0.31**	**0.35**	0.33	**0.34**	**0.26**	**0.33**	0.23	**0.15**	**0.16**	**0.08**	0.31	**0.12**	**0.09**
exoplayer	Qwen2.5-7B-Instruct	0.28	0.35	0.31	0.13	0.26	**0.70**	0.38	0.23	0.10	0.34	0.15	0.08	0.07	0.28	0.11	0.09
	LiteM	**0.50**	**0.46**	**0.47**	**0.36**	**0.44**	0.42	**0.43**	**0.32**	**0.87**	**0.46**	**0.60**	**0.62**	**0.66**	**0.33**	**0.44**	**0.46**
fastjson	Qwen2.5-7B-Instruct	0.08	**0.72**	0.14	0.11	0.07	**0.73**	0.13	0.08	0.04	0.18	0.07	-0.11	0.01	0.10	0.02	-0.08
	LiteM	**0.50**	0.23	**0.29**	**0.30**	**0.35**	0.21	**0.24**	**0.23**	**0.80**	**0.68**	**0.71**	**0.70**	**0.86**	**0.78**	**0.81**	**0.81**
flink	Qwen2.5-7B-Instruct	0.09	**0.40**	0.15	0.06	0.09	**0.63**	0.16	0.12	0.05	0.31	0.09	0.06	0.04	**0.35**	0.07	0.06
	LiteM	**0.27**	0.34	**0.30**	**0.25**	**0.28**	0.35	**0.31**	**0.27**	**0.19**	**0.36**	**0.17**	**0.18**	**0.51**	0.15	**0.14**	**0.19**
guava	Qwen2.5-7B-Instruct	0.38	0.40	0.39	0.11	0.32	**0.60**	0.42	0.18	0.15	**0.40**	0.22	0.11	0.09	0.36	0.15	0.06
	LiteM	**0.66**	**0.60**	**0.63**	**0.48**	**0.62**	0.58	**0.59**	**0.48**	**0.57**	0.39	**0.42**	**0.40**	**0.92**	**0.57**	**0.70**	**0.70**
jenkins	Qwen2.5-7B-Instruct	0.41	**0.58**	0.48	0.21	0.38	**0.71**	0.49	0.24	0.18	0.27	0.22	0.07	0.11	**0.32**	0.16	0.06
	LiteM	**0.64**	0.55	**0.59**	**0.44**	**0.56**	0.50	**0.53**	**0.37**	**0.31**	**0.40**	**0.35**	**0.24**	**0.15**	0.26	**0.18**	**0.11**
libgdx	Qwen2.5-7B-Instruct	0.09	**0.73**	0.17	0.12	0.08	**0.88**	0.15	0.09	0.04	0.32	0.07	0.02	0.05	0.29	0.09	0.02
	LiteM	**0.41**	0.32	**0.34**	**0.32**	**0.35**	0.31	**0.32**	**0.28**	**0.83**	**0.46**	**0.59**	**0.61**	**0.82**	**0.33**	**0.44**	**0.49**
logstash	Qwen2.5-7B-Instruct	0.13	**0.66**	**0.22**	**0.17**	0.15	**0.62**	0.24	0.12	0.06	**0.23**	0.09	0.01	0.11	**0.39**	0.17	**0.14**
	LiteM	**0.19**	0.10	0.12	0.08	**0.50**	0.27	**0.33**	**0.30**	**0.30**	0.18	**0.20**	**0.19**	**0.11**	0.32	**0.17**	0.14
mockito	Qwen2.5-7B-Instruct	0.13	**0.52**	0.21	0.10	0.17	**0.67**	0.27	0.11	0.07	**0.32**	0.11	0.04	0.09	**0.39**	**0.15**	0.13
	LiteM	**0.37**	0.25	**0.29**	**0.23**	**0.32**	0.30	**0.31**	**0.21**	**0.39**	0.22	**0.27**	**0.25**	**0.20**	0.23	0.14	**0.14**
openrefine	Qwen2.5-7B-Instruct	0.21	**0.64**	0.31	0.21	0.26	**0.77**	0.39	0.15	0.14	**0.51**	0.22	0.10	0.10	**0.43**	0.16	0.03
	LiteM	**0.45**	0.38	**0.40**	**0.34**	**0.49**	0.40	**0.44**	**0.31**	**0.54**	0.30	**0.36**	**0.34**	**0.33**	0.32	**0.31**	**0.26**
presto	Qwen2.5-7B-Instruct	0.13	**0.40**	0.20	0.11	0.23	**0.66**	**0.34**	0.13	0.12	0.31	0.17	0.06	0.09	0.27	0.13	0.07
	LiteM	**0.35**	0.33	**0.34**	**0.29**	**0.38**	0.31	0.34	**0.21**	**0.23**	**0.32**	**0.27**	**0.18**	**0.12**	**0.38**	**0.18**	**0.13**
quarkus	Qwen2.5-7B-Instruct	0.19	**0.61**	0.29	0.18	0.17	**0.69**	0.27	0.18	0.12	**0.45**	0.18	0.13	0.12	**0.44**	0.18	0.13
	LiteM	**0.42**	0.39	**0.40**	**0.33**	**0.45**	0.39	**0.41**	**0.35**	**0.22**	0.40	**0.29**	**0.24**	**0.22**	0.30	**0.20**	**0.16**
questdb	Qwen2.5-7B-Instruct	0.05	**0.56**	0.10	0.09	0.08	**0.72**	0.15	0.03	0.03	**0.31**	0.06	-0.01	0.03	**0.40**	0.06	0.04
	LiteM	**0.32**	0.23	**0.23**	**0.23**	**0.42**	0.25	**0.28**	**0.26**	**0.52**	0.22	**0.29**	**0.31**	**0.47**	0.28	**0.29**	**0.31**
redisson	Qwen2.5-7B-Instruct	0.05	**0.64**	0.09	0.10	0.06	**0.80**	0.11	0.14	0.12	0.52	0.19	0.23	0.15	0.58	0.24	0.11
	LiteM	**0.58**	0.42	**0.47**	**0.47**	**0.57**	0.33	**0.35**	**0.39**	**0.66**	**0.66**	**0.64**	**0.64**	**0.92**	**0.78**	**0.83**	**0.82**
rxjava	Qwen2.5-7B-Instruct	0.07	**0.57**	0.13	0.10	0.10	**0.70**	0.18	0.11	0.08	0.47	0.13	0.11	0.06	0.44	0.10	0.06
	LiteM	**0.59**	0.49	**0.51**	**0.50**	**0.59**	0.44	**0.48**	**0.47**	**0.87**	**0.63**	**0.72**	**0.73**	**0.73**	**0.45**	**0.53**	**0.55**
tink	Qwen2.5-7B-Instruct	0.25	0.29	0.27	0.07	0.20	0.51	0.29	0.04	0.11	0.23	0.15	0.05	0.04	**0.32**	0.07	0.06
	LiteM	**0.73**	**0.70**	**0.71**	**0.64**	**0.74**	**0.66**	**0.69**	**0.63**	**0.75**	**0.58**	**0.64**	**0.63**	**0.16**	0.31	**0.20**	**0.20**
Average	Qwen2.5-7B-Instruct	0.17	**0.54**	0.23	0.13	0.17	**0.71**	0.27	0.14	0.09	0.36	0.14	0.07	0.07	**0.36**	0.12	0.07
	LiteM	**0.45**	0.37	**0.39**	**0.34**	**0.45**	0.37	**0.39**	**0.33**	**0.49**	**0.39**	**0.39**	**0.38**	**0.43**	0.35	**0.33**	**0.32**

For Class, Method, and Block granularities, the Shapiro-Wilk test rejected the null hypothesis of normality for LiteM ($p < 0.05$), indicating non-normal distributions. To maintain a statistical methodology, we applied the non-parametric Wilcoxon signed-rank test to these remaining granularities. The tests confirmed that LiteM still outperforms with a statistically significant difference across all three granularities, with results of $p \approx 0.00029$ for Class, $p \approx 7.6 \times 10^{-6}$ for Method, and $p \approx 0.00071$ for Block. Then, with these results, we reject H_0 and accept H_1 for all granularities, showing statistical evidence that the LiteM model outperforms the LLM approach for unadmitted technical debt detection.

3.3 Absence of LLM Generation Hallucinations

As stated in section 2, Large Language Models tend to suffer from hallucinations in their responses. It was expected that the LLM might generate responses outside the "yes" or "no" response constraints, even though a strict two-token limit was enforced in the execution framework. However, this was not the case: the model successfully restricted its answers to the expected options across the entire dataset. This indicates that the model encountered no difficulties in comprehending the task boundaries or adhering to formatting constraints. Therefore, the lower evaluation metrics reported in Section **??** cannot be partially explained due to parsing failures; instead, they purely reflect a limitation in the LLM's capability to accurately classify Technical Debt from raw, mainly fine-grained source code.

4 Reproducibility

This section presents the artifacts made available to reproduce and replicate this experiment. All source code is hosted on GitHub[4], **which includes a detailed step-by-step `README.md` configuration and execution guide.** Furthermore, all the prompts and inference results generated by Qwen2.5-7B-Instruct during the execution pipeline are publicly available on Google Drive[5]. The LLM API parameters used in this experiment are also made available on GitHub.

4.1 Requirements

To reproduce this exact experiment, access to the Qwen2.5-7B-Instruct[6] model is required via an API endpoint **(or an equivalent OpenAI-compatible API)**. A machine with at least 8GB of RAM is recommended to execute the client-side Python pipeline, whereas hosting and running the LLM locally requires significantly greater computational resources. The pipeline scripts were implemented entirely in Python **and manage dependencies via a standard virtual environment setup (`requirements.txt`)**. While Qwen was used for this study, any other Large Language Model (LLM) API can be substituted.

4.2 UTD Dataset and LiteM Execution

The original dataset and execution scripts for Yu et al.'s LiteM approach [10] are available in the authors' GitHub repository[7]. Our experiment pipeline scripts are designed to automatically download this dataset if it is not already present locally.

4.3 Non-Deterministic Inference Results

Although a temperature of 0 was used, the Large Language Model (LLM) inference results may still exhibit variations. This nondeterministic behavior typically occurs due to floating-point arithmetic optimizations (such as atomic operations and reduction algorithms) executed across highly parallelized GPU architectures. Because the order of parallel addition operations can vary slightly between runs, minor rounding errors can accumulate, occasionally altering the token probabilities and leading to different text generation paths.

5 Threats to Validity

This section presents the threats to validity associated with this study. First, regarding internal validity, our evaluation is limited to a single Large Language Model: Qwen2.5-7B-Instruct. Consequently, this model may not be fully representative of all LLMs, particularly given its relatively small parameter size.

Secondly, due to significant time and resource constraints, we were unable to evaluate alternative prompting techniques, such as few-shot or Chain-of-Thought (CoT) strategies. Testing these diverse prompting paradigms remains an important direction for future work to determine if they might mitigate the observed performance gaps.

Lastly, concerning construct validity, it is important to note that Yu et al.'s [10] dataset may contain false negative labels regarding general debt; since their original proposal specifically aimed to identify UTD, not all latent TD cases may be identified by their annotation framework.

6 Conclusion

This study evaluated the performance of a Large Language Model (LLM) against a lightweight, metric-based machine learning approach for detecting Technical Debt (TD) in code snippets without the aid of source code comments. Specifically, we benchmarked the *Qwen2.5-7B-Instruct* model against the *LiteM* model proposed by Yu et al. [10] across four distinct code granularities. The results demonstrate that despite the LLM's capability to leverage the entire raw source code as context, the metric-based LiteM model delivers a superior overall classification performance across all evaluated granularity levels.

Our formal statistical analysis, leveraging normality verification and hypothesis testing, confirms that LiteM's higher performance over the LLM is significant ($p < 0.01$) across all four code granularities. Our analysis also reveals that while the LLM demonstrated great capacity to comprehend the task boundaries and output constraints ("yes" and "no"), its overall classification performance was limited. This performance gap can be partially explained by the model's tendency to flag minor code quality issues as Technical Debt, leading to an excessive number of false positives.

These findings suggest that traditional engineered code metrics remain highly effective and statistically superior indicators for technical debt compared to a zero-shot LLM approach. In future work, we plan to investigate whether advanced prompting techniques—such as few-shot learning or Chain-of-Thought paradigms—or the evaluation of larger parameter models can bridge this performance gap and better leverage the semantic context of source code.

[4]https://github.com/MiguelOlivRd/Unadmitted-Technical-Debt-Detection-with-LLM

[5]https://drive.google.com/drive/folders/1uYRE-UERaJJFzqe3L6EUYwO8mNlTwfLN?usp=sharing

[6]https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

[7]https://github.com/HduDBSI/Dataset4TD

References

[1] Abdulaziz Alhefdhi, Hoa Khanh Dam, Yusuf Sulistyo Nugroho, Hideaki Hata, Takashi Ishio, and Aditya Ghose. 2022. A framework for conditional statement technical debt identification and description. *Automated Software Engineering* 29, 2 (03 Oct 2022), 60. doi:10.1007/s10515-022-00364-8

[2] Shaiful Chowdhury, Hisham Kidwai, and Muhammad Asaduzzaman. 2025. Evidence is All We Need: Do Self-Admitted Technical Debts Impact Method-Level Maintenance?. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 813–825. doi:10.1109/MSR66628.2025.00118

[3] Ward Cunningham. 1992. The WyCash portfolio management system. *SIGPLAN OOPS Mess.* 4, 2 (Dec. 1992), 29–30. doi:10.1145/157710.157715

[4] Yikun Li, Mohamed Soliman, Paris Avgeriou, Jie Tan, and Jiakun Liu. 2026. Text Tells the Cost: Predicting and Analyzing Repayment Effort of Self-Admitted Technical Debt. arXiv:2309.06020 [cs.SE] https://arxiv.org/abs/2309.06020

[5] Everton da Silva Maldonado, Emad Shihab, and Nikolaos Tsantalis. 2017. Using Natural Language Processing to Automatically Detect Self-Admitted Technical Debt. *IEEE Transactions on Software Engineering* 43, 11 (2017), 1044–1062. doi:10.1109/TSE.2017.2654244

[6] Everton da Silva Maldonado, Emad Shihab, and Nikolaos Tsantalis. 2017. Using Natural Language Processing to Automatically Detect Self-Admitted Technical Debt. *IEEE Transactions on Software Engineering* 43, 11 (2017), 1044–1062. doi:10.1109/TSE.2017.2654244

[7] Antonio Mastropaolo, Massimiliano Di Penta, and Gabriele Bavota. 2024. Towards Automatically Addressing Self-Admitted Technical Debt: How Far Are We?. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering* (Echternach, Luxembourg) *(ASE '23)*. IEEE Press, 585–597. doi:10.1109/ASE56229.2023.00103

[8] Mohammad Sadegh Sheikhaei, Yuan Tian, Shaowei Wang, and Bowen Xu. 2026. A Large-Scale Empirical Evaluation of LLMs for Automated Self-Admitted Technical

Debt Repayment. *ACM Trans. Softw. Eng. Methodol.* (Feb. 2026). doi:10.1145/3796704 Just Accepted.

[9] Dimitrios Tsoukalas, Nikolaos Mittas, Alexander Chatzigeorgiou, Dionysios Kehagias, Apostolos Ampatzoglou, Theodoros Amanatidis, and Lefteris Angelis. 2022. Machine Learning for Technical Debt Identification. *IEEE Transactions on Software Engineering* 48, 12 (2022), 4892–4906. doi:10.1109/TSE.2021.3129355

[10] Dongjin Yu, Yihang Xu, Xin Chen, Quanxin Yang, and Sixuan Wang. 2025. Unadmitted Technical Debt: Dataset and Detection Approaches. *IEEE Transactions on Software Engineering* 51, 12 (2025), 3608–3631. doi:10.1109/TSE.2025.3623644

[11] Fiorella Zampetti, Cedric Noiseux, Giuliano Antoniol, Foutse Khomh, and Massimiliano Di Penta. 2017. Recommending when Design Technical Debt Should be Self-Admitted. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 216–226. doi:10.1109/ICSME.2017.44